Where each objective is implemented

Project: Handwritten Modi Character Recognition using Handcrafted Features (Group TY-A2)

All paths are inside the project folder `modi-lipi-xai`. Every file listed below also has a line at the top saying which objective it belongs to (search the code for "(synopsis)").

Objective 1: Recognise 57 Modi characters using HOG, Zernike moments and LBP instead of raw pixels

Base papers: Kulkarni et al. 2014, Mandke et al. 2024, Anam & Gupta 2015

What	Where
Cleaning with Otsu thresholding	`src/preprocess.py`, `binarize()` (line 121)
HOG features (stroke direction, 1,764 values)	`src/features/hog_features.py`, `extract()` (line 48)
Zernike moments (overall shape, 49 values)	`src/features/zernike_features.py`, `extract()` (line 43)
LBP features (small local patterns, 448 values)	`src/features/lbp_features.py`, `extract()` (line 76)
Joining the three into one 2,261-value vector	`src/features/fusion.py`, `extract_one()` (line 89)
Testing what each feature adds (ablation)	`scripts/run_training.py`, `run_ablation()` (line 191)
Command	`python scripts\run_training.py --only ablation`
Result file	`results/metrics/ablation.json`
In the app	Results page, Features tab

What we got (SVM, 500 training images per character): HOG alone 83.6%, LBP alone 79.3%, Zernike alone 54.0%, all three together 86.4%. Taking one out: without HOG 81.4%, without LBP 83.9%, without Zernike 86.0%. So HOG matters most, LBP adds a clear gain, and Zernike adds very little once the other two are in.

Objective 2: Compare Random Forest, SVM and XGBoost, and check whether stacking them helps

Base paper: Ekbote et al. 2022

What	Where
The three models	`src/models.py`, `make_base()` (line 89)
Stacking ensemble	`src/models.py`, `make_stack()` (line 100)
Training everything together	`src/models.py`, `train()` (line 154)
Running the comparison	`scripts/run_training.py`, `run_main()` (line 107)
Command	`python scripts\run_training.py --only main`

Result files	`results/metrics/model_comparison.json`, `results/figures/model_comparison.png`
In the app	Results page, Models tab

What we got (11,400 test images): SVM 86.4%, stacking 84.8%, Random Forest 84.0%, XGBoost 83.8%. Stacking did not beat the SVM on its own, and we report that as it is.

Objective 3: Use SHAP to show which part of a character the model looked at, and list the most confused pairs

Base papers: Kolte & Gupta 2025, Anam & Gupta 2015

What	Where
Explaining one prediction	`src/explain.py`, `GroupedExplainer.explain()` (line 142)
Overall importance of each feature and region	`src/explain.py`, `run_global()` (line 251)
Explaining the confused pairs	`src/explain.py`, `contrast_pairs()` (line 365)
Finding the most confused pairs	`src/evaluate.py`, `most_confused_pairs()` (line 58)
Used by the app for any uploaded character	`src/predict.py`, `Recognizer.explain()` (line 205)
Command	`python scripts\run_evaluation.py`
Result files	`results/metrics/most_confused_pairs.json`, `results/shap/global_importance.json`, `results/shap/confused_pairs_contrast.json`
In the app	Read page (tap a character, then "Explain with SHAP"), Explain page, Results page Errors tab

What we got: the most confused pairs are i and ii (91 test images), h and thha (89) and p and thha (54). HOG gives about 66% of the evidence, LBP 31% and Zernike 3%. To check the explanations are real, we removed the 3 regions SHAP ranked highest: the model's confidence dropped by 0.48 on average, against 0.06 when we removed 3 random regions.

Objective 4: Compare our model with a simple CNN trained on the same small amounts of data

Base papers: Bhong & Kulkarni 2025, Chandankhede et al. 2025, Mandke et al. 2024

What	Where
The simple CNN	`src/baselines/cnn_baseline.py`, `PlainCNN` (line 52)
Training and testing it	`src/baselines/cnn_baseline.py`, `train_and_evaluate()` (line 246)

Same training and test images for both models	`src/data_loader.py`, `stratified_split()` (line 440)
Same scoring for both models	`src/learning_curve.py`, `score_predictions()` (line 41)
Classical models at each data size	`scripts/run_training.py`, `run_curve()` (line 251)
CNN at each data size	`scripts/run_cnn_baseline.py`, `main()` (line 82)
Commands	`python scripts\run_training.py --only curve` and `python scripts\run_cnn_baseline.py`
Result files	`results/metrics/learning_curve_classical.json`, `results/metrics/learning_curve_cnn.json`
In the app	Results page, Curve tab (also the small chart on the Home page)

What we got (best classical model vs CNN): 72 images per character 80.4% vs 72.3%, 90 images 81.2% vs 77.8%, 250 images 84.5% vs 85.1%. So our model does better when data is small, and the CNN catches up somewhere between 90 and 250 images per character. The larger sizes are still training.